

# Assignment – 4

## Software Testing, CI/CD Automation & Cloud Observability

*SIMATS School of Engineering | Department of Computer Science & Engineering*

|             |                      |
|-------------|----------------------|
| Name        | CH. Chakradhar       |
| Reg. no     | 192365043            |
| Course Name | Software Engineering |
| Course Code | CSA1001              |

### Question 1: Mutation Testing and Fuzz Testing

**Prompt:** Explain Mutation Testing and Fuzz Testing. How do these testing techniques enhance the reliability and security of software systems? [7 Marks, CO4]

#### 1. Mutation Testing

Mutation Testing is an advanced, fault-based white-box testing technique designed to evaluate the adequacy and effectiveness of an existing test suite[cite: 7]. Instead of assessing whether application code passes tests, mutation testing evaluates whether the test suite is powerful enough to catch deliberately introduced syntactic defects (termed mutants)[cite: 7].

- **Mechanism of Operation:** A mutation engine parses source code into an Abstract Syntax Tree (AST) and introduces small syntactic modifications using predefined mutation operators—such as altering relational operators (e.g., changing `a > b` to `a >= b`), mutating arithmetic operators (changing `+` to `-`), or modifying conditional boundaries and return statements[cite: 7].
- **Mutant Evaluation:** The existing test suite is executed against each mutated program[cite: 7]. If a test case fails, the mutant is marked as **killed** (demonstrating test adequacy)[cite: 7]. If all test cases pass without detecting the mutant, the mutant has **survived**, revealing an untested edge case, missing assertion, or blind spot in the test suite[cite: 7].
- **Mutation Score Metric:** The quality of test coverage is formulated quantitatively as:  
$$\text{Mutation Score} = (\text{Killed Mutants} / \text{Total Non-Equivalent Mutants}) \times 100\%[\text{cite: 7}]$$

#### 2. Fuzz Testing (Fuzzing)

Fuzz Testing is a dynamic, automated black-box or grey-box security testing technique that supplies invalid, malformed, unexpected, or randomized input streams to an application to trigger crashes, memory leaks, assertion violations, or unhandled exceptions[cite: 7].

- **Mechanism of Operation:** A fuzzer (e.g., AFL++, libFuzzer) injects millions of mutated binary payloads, malformed JSON strings, boundary integers, or corrupted protocol packets into system entry points (network sockets, CLI parsers, file ingestion pipelines)[cite: 7].
- **Coverage-Guided Fuzzing:** Advanced fuzzers instrument binaries during compilation to observe branch execution[cite: 7]. When an input triggers execution of a previously unvisited code path, that input is retained in the corpus to generate subsequent targeted mutations[cite: 7].

| Comparison Dimension   | Mutation Testing                                                    | Fuzz Testing                                                           |
|------------------------|---------------------------------------------------------------------|------------------------------------------------------------------------|
| Testing Classification | White-box test suite quality evaluation technique[cite: 7].         | Black-box / Grey-box dynamic vulnerability discovery[cite: 7].         |
| Primary Objective      | Identifies blind spots and weak assertions in test suites[cite: 7]. | Discovers buffer overflows, memory leaks, panics, and CVEs[cite: 7].   |
| What Gets Mutated      | Application source code / AST statements[cite: 7].                  | Input data streams and payload structures[cite: 7].                    |
| Success Criterion      | High mutation score (maximum mutants killed by tests)[cite: 7].     | System resilience (zero unexpected crashes, halts, or leaks)[cite: 7]. |

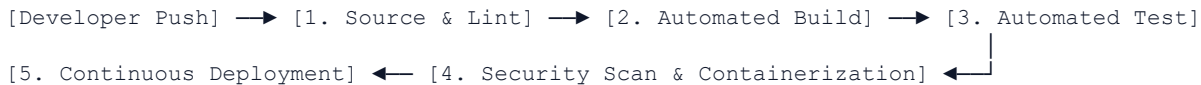
3. How These Techniques Enhance Software Reliability and Security

- **Eliminating False Confidence from Line Coverage:** Traditional code coverage metrics indicate whether a line was executed, not whether its behavior was verified[cite: 7]. Mutation testing guarantees that tests contain meaningful, defensive assertions capable of detecting regressions, directly improving system reliability[cite: 7].
- **Uncovering Zero-Day Security Vulnerabilities:** Fuzz testing probes software against unconventional inputs that manual testers rarely conceive, unearthing critical security flaws like buffer overflows, integer underflows/overflows, format string injections, and Denial of Service (DoS) conditions[cite: 7].
- **Hardening System Boundary Defenses:** Combining mutation testing (which validates internal logical branches) with fuzzing (which exercises external input ingestion) delivers defense-in-depth, ensuring both internal logic correctness and resilient boundary validation[cite: 7].

## Question 2: Key Stages of a CI/CD Pipeline & Role of Docker

**Prompt:** Describe the Key Stages of a CI/CD Pipeline (using Jenkins or GitHub Actions). How does using Docker for containerization support continuous delivery? [7 Marks, CO4]

### 1. Key Stages of an Enterprise CI/CD Pipeline



- **Stage 1: Source Checkout, Linting & Formatting:** Triggers on pull requests or pushes to main[cite: 7]. Clones the repository, validates syntax, and enforces formatting standards using tools such as ESLint, Prettier, or SonarLint to ensure stylistic consistency[cite: 7].
- **Stage 2: Compile & Build:** Compiles source code into deployable artifacts (e.g., transpiling TypeScript, bundling React assets, or compiling Go/Java binaries) while leveraging caches to accelerate build execution[cite: 7].
- **Stage 3: Automated Testing:** Executes unit tests, integration tests, and contract verification suites (e.g., Jest, PyTest, JUnit)[cite: 7]. If any test fails or code coverage drops below threshold, the pipeline immediately halts[cite: 7].
- **Stage 4: Security Analysis & Container Image Packaging:** Executes Static Application Security Testing (SAST) and dependency vulnerability scans (Trivy, Snyk)[cite: 7]. Docker builds the production image, tags it with the Git commit SHA, and pushes it to a container registry[cite: 7].
- **Stage 5: Continuous Deployment / Delivery:** Deploys the containerized artifact to staging or production Kubernetes clusters using zero-downtime rolling updates, canary rollouts, or blue/green strategies[cite: 7].

## 2. Production GitHub Actions Pipeline Definition Example

```

name: Production CI/CD Pipeline
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  build-test-deliver:
    runs-on: ubuntu-latest
    steps:
      - name: 1. Checkout Source Code
        uses: actions/checkout@v4

      - name: 2. Setup Node.js Runtime
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: 3. Install Dependencies & Run Linting
        run: |
          npm ci
          npm run lint

      - name: 4. Execute Automated Test Suites
        run: npm test -- --coverage --ci

      - name: 5. Build and Push Docker Image
        if: github.ref == 'refs/heads/main'
        run: |
          echo "${{ secrets.DOCKER_PASSWORD }}" | docker login -u "${{ secrets.DOCKER_USERNAME }}" -
          docker build -t myorg/app-backend:${{ github.sha }} .
          docker push myorg/app-backend:${{ github.sha }}

```

## 3. How Docker Containerization Supports Continuous Delivery

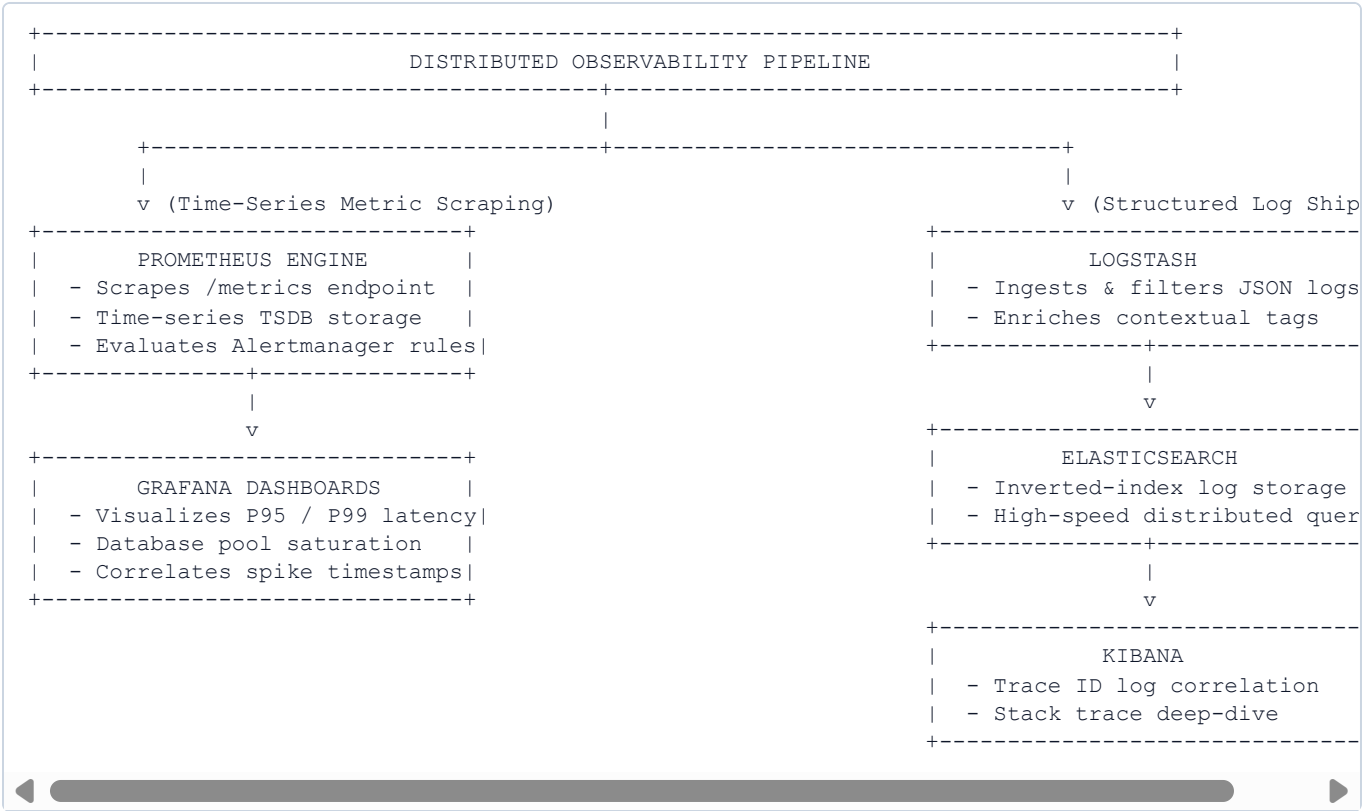
- **Elimination of Configuration Drift:** Containers encapsulate the application bytecode, system packages, OS binaries, and runtime settings together[cite: 7]. This guarantees that the exact environment tested in CI behaves identically in staging and production, solving the "works on my machine" dilemma[cite: 7].
- **Immutable and Auditable Deployment Artifacts:** Instead of executing fragile, in-place installations on production virtual machines, Docker packages immutable container images tagged by commit hash[cite: 7]. Rollbacks to previously known good states take seconds[cite: 7].
- **Micro-Isolation and Fast Horizontal Scaling:** Containerized microservices run in lightweight process namespaces with bounded CPU/memory quotas, allowing CI/CD runners to spin up ephemeral testing environments on demand and deploy to orchestrators like Kubernetes without side effects[cite: 7].

Question 3: Case Study – Intermittent Slowdowns in a Large SaaS Application

**Prompt:** Case Study – Intermittent Slowdowns in a Large SaaS Application: Explain how monitoring and logging tools such as Prometheus, Grafana, and the ELK Stack can help detect, diagnose, and resolve performance issues. [6 Marks, CO4]

1. Incident Context & Observability Architecture

In a high-throughput multi-tenant SaaS application, intermittent performance degradation (e.g., transient response latency spikes from 120ms to 4500ms, intermittent HTTP 504 gateway timeouts) cannot be diagnosed through manual log checks[cite: 7]. Resolving these anomalies requires a unified observability framework combining metrics aggregation (Prometheus), real-time telemetry visualization (Grafana), and centralized distributed log analysis (ELK Stack)[cite: 7].



2. Stage-by-Stage Troubleshooting: Detect, Diagnose, and Resolve

| Phase                  | Tool Utilized             | Technical Mechanism & Concrete Actions                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------------------------|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Detection (Phase 1) | Prometheus + Alertmanager | <ul style="list-style-type: none"><li>• Prometheus scrapes time-series metrics from microservices at 15-second intervals via /metrics[cite: 7].</li><li>• PromQL query evaluates high percentiles:<br/>histogram_quantile(0.99, sum(rate(http_request_duration_seconds_bucket[5m])) by (le)) &gt; 2.0[cite: 7].</li><li>• Alertmanager fires automated PagerDuty/Slack alerts the moment P99 latency exceeds SLA thresholds for more than 2 minutes[cite: 7].</li></ul> |

| Phase                                             | Tool Utilized                                         | Technical Mechanism & Concrete Actions                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|---------------------------------------------------|-------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>2. Diagnosis (Phase 2)</b>                     | <b>Grafana</b>                                        | <ul style="list-style-type: none"><li>• On-call engineers inspect Grafana telemetry dashboards correlating application metrics with infrastructure metrics[cite: 7].</li><li>• By analyzing CPU, memory, database connection pool exhaustion, and garbage collection pauses, the team identifies that the latency spike matches database connection pool exhaustion and slow search query spikes[cite: 7].</li></ul>                                                                 |
| <b>3. Root-Cause Analysis (Phase 3)</b>           | <b>ELK Stack</b><br>(Elasticsearch, Logstash, Kibana) | <ul style="list-style-type: none"><li>• Logstash ingests structured JSON application logs enriched with <code>correlation_id</code> and <code>tenant_id</code>[cite: 7].</li><li>• In Kibana, engineers filter logs for the spike timestamp: <code>status:504 AND latency:&gt;3000ms</code>[cite: 7].</li><li>• Pinpoints an unindexed full-table scan triggered by a tenant bulk-export report that locked connection threads and queued upstream HTTP requests[cite: 7].</li></ul> |
| <b>4. Resolution &amp; Verification (Phase 4)</b> | <b>Prometheus &amp; Grafana</b>                       | <ul style="list-style-type: none"><li>• Engineers apply a database index, route heavy analytic queries to a read-replica, and scale up connection pool limits[cite: 7].</li><li>• Grafana dashboards show real-time latency dropping back to baseline (&lt;120ms), while Prometheus resolves the active alert[cite: 7].</li></ul>                                                                                                                                                    |